

Optimal Motion Planning Considering Task Redundancy in Trajectory Tracking Applications for Industrial Robot

Wenxiang Gao, Qing Tang, *Member, IEEE*, and Jin Yao

Abstract—Task redundancy which is defined as that a robotic manipulator has more degrees of freedom than those strictly required to execute a given task exists in most industrial applications. This paper focuses on how to automatically generate a collision-free motion trajectory for an industrial robot with task redundant applications in a complex environment. The task redundant space is divided into grid for trajectory tracking motion planning. The expanding process map the joint space to the task redundant space for neighborhood searching. The cost evaluation not only calculates the distance in task redundant space but also considers the cost defined in joint space including both singularity and joint limits. The simulation experiments verify that our proposed algorithm not only could realize a collision-free motion and handle the singularity and joint limits, but also be able to track complex end effector trajectory within an obstacle-intensive environment. The results show that the proposed algorithm could be used in all kinds of task redundant trajectory applications in industry.

Index Terms—Task redundancy, collision-free motion planning, optimality, singularity, joint limits.

I. INTRODUCTION

Recent years, robots play an increasingly important role in manufacturing industries, especially with the hazardous tasks such as welding [1], polishing [2], spraying [3], etc. However, the increasingly complicated trajectories and complex workspace with dense obstacles make the automatic path planning much more difficult [4].

Task redundancy commonly exists in the industrial applications [5]. The degree of freedom of task redundancy varies with the specific task. The rotation of the tool around its symmetrical axis is redundant for most applications such as welding, polishing, and spraying. Thus there is at least 1-DOF of redundancy when executing the task. Besides rotation along the symmetrical axis of the tools, rotation around the axis in the tangent plane of the workpiece is also usually considered in the industrial applications such as the inclined welding and the oblique angle spray [6]. Different from the redundancy caused by symmetry axis, most of the time the redundancy caused by skew angle should be limited to a certain threshold in order to ensure the product quality [7]. In these circumstances, the dimension of the task redundancy would increase to two. Further more, considering the orientation of the torch might be totally undefined in some welding applications is also uncertain [8],

the dimension of the task redundancy might be even higher. How to generate feasible motion for industrial robot in task redundancy space is an urgent problem to be solved in the application of robot in modern industry.

Motion planning algorithms which output feasible robot motion for complex environment is widely studied [9]. Different algorithms have been proposed to avoid the potential collision and achieve a minimum trajectory cost [10], such as Combinatorial Roadmaps [11], Artificial Potential Field (APF) [12]. However, the Combinatorial Roadmaps needs explicit obstacle information and the APF method would easily be trapped with local minimum and usually be implemented as the low level control. Besides the above two algorithms, grid search algorithms and sampling-based algorithms are the most popular motion planning algorithms nowadays. The grid search algorithm is the most straightforward form of path planning. The configuration space is directly rasterized. Once an initial-goal query is given, the graph search algorithms, such as A*, best-first or bidirectional search, are used to find the optimal path [13]. The A* algorithm which consists of a heuristic cost could generate an optimal trajectory in the grid efficiently. Lots of improvements are proposed to enhance the A* algorithm for lifelong planning [14], dynamic environment [15], anytime style [16], etc. On the other hand, sampling-base algorithms which include the PRM [17], RRT [18] and their variants have to construct a roadmap or tree using sampling techniques first, then adopting different graph search algorithms according to the query form. Fortunately, most of the variants of the classical PRM and RRT, namely PRM*, RRT*, FMT*, RRTx, etc., are proved to be probabilistic complete and asymptotical optimal. However resolution complete rather than probabilistically complete is required for industrial applications. The grid search algorithm introduced above could easily obtain the resolution completeness according to its inherent rasterizing characteristics. LaValle *et al.* proves that some forms of grid search yield performance that is comparable to the sampling-based algorithm [19]. This makes the grid search algorithms a perfect choice to solve the robot motion planning problems for industrial applications.

The remainder of this paper is organized as follows. The optimal motion planning algorithm is proposed in Section II which includes the algorithm framework, the cost function definition and the expanding strategy. Simulation experiments are carried out in Section III, which verify the ability of singularity and joint limit avoidance of the algorithm and its feasibility in 1D and 2D environment. Finally, the conclusion is presented in Section IV.

*This work was supported by Sichuan Key Research and Development Plan under Grant 2020YFG0116 and Grant 2021YFG0074. (Corresponding author: Qing Tang.).

The authors are with the School of Mechanical Engineering, Sichuan University, Chengdu 610065, China (E-mail: gao_scu@163.com; tangqing_scu@163.com; jinyao163@163.com).

II. OPTIMAL COLLISION-FREE MOTION PLANNING

In this section, a modified A* algorithm is introduced to solve the optimal collision-free motion planning problem of the industrial robot for the task redundant applications. The following sections would introduce the framework of the algorithm, the cost evaluation function, expanding strategy and the proof of optimality respectively.

A. The Framework of the Modified A* Algorithm

Algorithm 1: The modified A* motion planning algorithm

```

1 Initialization:  $S_{closed} \leftarrow \emptyset, S_{open} \leftarrow \{p_{init}\};$ 
2 while  $S_{open} \neq \emptyset$  do
3    $p_{min} \leftarrow \arg \min \{f(p) | p \in S_{open}\};$ 
4   if  $p_{min} \notin p_{goal}$  then
5      $\{S_{open}, S_{closed}\} =$ 
        $Expand(S_{open}, S_{closed}, p_{min});$ 
6   else
7      $q \leftarrow SearchPath(S_{closed}),$  break;
8   end
9 end

```

The scheme of the modified A* motion planning algorithm is shown in Algorithm 1. The robot configuration which is defined in joint space could be denoted as $p = \{\theta_i\}$. The corresponding end effector could be represented as $\{P, R\}$ in Cartesian space. The two representations of the configuration p are transformed by the forward and inverse kinematics of the robot. The algorithm maintains two sets: The OPEN set S_{open} holds all the configurations to be explored and the CLOSED set S_{closed} contains all the configurations which have been explored. In the initialization stage, both sets are set empty and the initial configuration is inserted to the OPEN set (Algorithm 1 line 1). Then the A* algorithm repeatedly explores the configurations in the OPEN set until the OPEN set is empty (Algorithm 1 line 2-6). The exploration sequence is based on the cost evaluation of the configuration and the OPEN set uses a priority queue for improving the efficiency. Once the exploring configuration reaches the goal configurations, the motion planning algorithm terminates and an optimal path could be found by backtracking the configurations from the end to the start (Algorithm 1 line 7).

B. Cost Evaluation Function

It is well known that the A* algorithm achieves better performance by using heuristics to guide its expanding. No A*-like algorithm could expand fewer nodes than A* if the heuristic function is consistent. In this paper, not only the heuristic function is included in the cost evaluation, but also a guiding function which evaluate the singularity and joint limits of the whole trajectory is appended. The overall cost evaluation is defined as follows:

$$f(p) = \omega_g g(p) + \omega_h h(p) + \omega_e e(p) \quad (1)$$

where the function $g(p)$ denotes the actual cost from the start configuration to the current configuration, the function $h(p)$ denotes the heuristic cost from the current configuration to the goal configurations. The guiding function $e(p)$ is designed to avoid the movement of robot toward the singularity and joint boundaries. The coefficients $\omega_g, \omega_h, \omega_e$ are the weights of the corresponding cost functions.

1) *Actual Cost and Heuristic Cost:* In our proposed algorithm, the actual cost which consists of the distances in translation and rotation is defined in an incremental form. The actual cost of the successor is the sum of the actual cost of the ancestor and the actual cost between the ancestor and successor. The equation (2) denotes the iteration calculation of the actual cost.

$$g(p_{suc}) = g(p_{anc}) + \omega_t cost_t(p_{anc}, p_{suc}) + \omega_r cost_r(p_{anc}, p_{suc}) \quad (2)$$

where $g(p_{anc})$ and $g(p_{suc})$ are the actual cost of the ancestor configuration and the successor configuration respectively. $cost_t(p_{anc}, p_{suc})$ and $cost_r(p_{anc}, p_{suc})$ are the translational cost and the rotational cost between the ancestor configuration and the successor configuration respectively. ω_t and ω_r are the weights of the translational and rotational cost.

The heuristic cost of the current configuration which estimates the cost from the current configuration to the goal is calculated as the instant cost from the two configurations rather than an accumulative cost like the actual cost. Equation 3 shows the definition.

$$h(p) = \omega_t cost_t(p, p_{goal}) + \omega_r cost_r(p, p_{goal}) \quad (3)$$

The cost evaluation could refer to the metric of the task redundant space. The metric could be divided into translational metric and rotational metric. The task redundant space usually consists of one-dimensional translational space and n-dimensional rotational space depending on the task redundancy. Since the one-dimensional translational space is the space along the working trajectory, the Cartesian representation of the configuration is adopted in the cost evaluation. The translational cost is defined as the length of the working trajectory as follows:

$$cost_t(p_j, p_k) = \sum_{i=j}^{k-1} \|P_{i+1} - P_i\| \quad (4)$$

where the trajectory is divided into small segments. The length of each segment is measured by the Euclidean distance. The overall length of the trajectory is the sum of all segment lengths along the trajectory.

The rotational cost definition is different from translational cost because the rotational space is an n-dimensional special orthogonal group. The orientation matrix of the end-effector R_i is adopted as the configuration p_i 's representation in the rotational cost representation. The rotation matrix between two configurations could be defined by the relative rotation matrix $R_i^j = R_i^T R_j$. Thus the rotational cost for the modified

A* algorithm could be defined as the angle rotated along the rotational axis, as follows:

$$cost_r(p_i, p_j) = \|\theta_i^j\| = \arccos\left(\frac{\text{trace}(R_i^j) - 1}{2}\right) \quad (5)$$

where the rotational cost definition could be considered as the angle rotated around a unit axis from frame i to frame j . From the definition, we could notice that the translational cost would always be positive for different configurations, and would be zero if and only if the positions are the same. The rotational cost would be non-negative for all rotations in n -dimensional special orthogonal group.

2) *Guiding Cost Related to Joint Limits and Singularity:* Different from the traditional A* algorithm, not only the obstacles occupy the infeasible configurations, but also the over limits of joint angles and singular configurations are included in the infeasible configurations. What's more, the singularity and joint limits are not defined by the end effector but the joints of the robot. Although the original A* algorithm is capable of avoiding the joint limits and singularity by considering the related configurations as the obstacle occupied configurations. However, keep a certain distance from the joint boundaries and singularity still sounds a nice solution to the robot. In this paper, we introduce a guiding function which works as part of the cost during planning to guide the planner away from the joint boundaries and singularity. The guiding function $e(p)$ introduced in equation (1) could be defined as follows:

$$e(p_i) = \max \left\{ \begin{array}{l} e(p_{i-1}), \\ \omega_l cost_l(p_i) + \omega_s cost_s(p_i) \end{array} \right\} \quad (6)$$

where $cost_l(p)$ is the joint limits cost evaluation function and $cost_s(p)$ is the singularity cost evaluation. ω_l and ω_s are the weights for each evaluation functions respectively. It could be noted from the formula (6) that the guiding function calculates the maximum cost of all configurations throughout the whole trajectory, thus it is used as the index for the evaluation of the entire trajectory.

In order to keep the joint limit cost function consistent and with explicit physical meanings, we define the cost with the sigmoid function as follows:

$$cost_l(p) = \sum_{i=1}^n \frac{k_{l1}}{1 + \exp\left[k_{l2} \frac{(\theta_{i,max} - \theta_i)(\theta_i - \theta_{i,min})}{(\theta_{i,max} - \theta_{i,min})^2}\right]} \quad (7)$$

where θ_i is the i^{th} joint of the serial industrial robot under the configuration p . $\theta_{i,min}$ and $\theta_{i,max}$ are the lower and upper boundaries of the i^{th} joint. n is the number of robot joints. Two parameters k_{l1} and k_{l2} represent the maximum cost of over limits and the rates of cost change near the boundaries respectively.

Singularity evaluation is modified from the manipulability measure proposed by Yoshikawa [20]. Similar to the joint

limits evaluation, we combined the manipulability with sigmoid function which is almost zero in ordinary configuration but reaches up to maximum near the singularity. Equation (8) shows our evaluation function, in which k_{s1} and k_{s2} represent the maximum value of the singularity cost and the rate of manipulability measure change near singularity respectively. The parameter h is the threshold of the singularity.

$$cost_s(p) = \frac{k_{s1}}{1 + \exp\left[k_{s2} \left(\sqrt{\det(\mathbf{J}\mathbf{J}^T)} - h\right)\right]} \quad (8)$$

where $\mathbf{J}$ is the Jacobian matrix of the current robot configuration.

C. Expanding Strategy

Line 3 in Algorithm 1 indicates that the minimum cost node is preferentially chosen in each iteration. Then the OPEN set and the CLOSED set are updated according to the expanded node's neighbors as Line 5 in Algorithm 1. The specific expanding procedures are explained in Algorithm 2.

Firstly the position and orientation of the end effector is calculated by the forward kinematic of the robot (Algorithm 2, line 1). Then the neighborhood of the end effector could easily be calculated from the grid by simply finding the adjacent grid in task redundant space. Let $S_{neighbor}(T_{EE})$ denotes the neighbor set of the current robot configuration in the task redundant space (Algorithm 2, line 2). For each end-effector configuration in the neighbor set, the corresponding joint configuration is calculated by using the numerical iterative methods introduced in [21]. The current joint configuration is used as the initial value for the iteration (Algorithm 2, line 3), therefore the joint trajectory could be continuous. Most of the time, the inverse kinematics could be calculated very quickly since the target end-effector configuration is in the neighborhood of the initial iteration configuration. However the convergence speed might slow down due to the singularity of the robot configuration. Because the singularity is taken into consideration in our proposed planning algorithm as the cost evaluation, the generated trajectory could automatically deviate from the singular configuration to achieve a relatively fast convergence speed of the inverse kinematics.

The successor configuration is first searched in the OPEN set and CLOSED set. The configuration would be skipped if it has been in the CLOSED set or its overall cost and actual cost would be set to infinit if it could not be found in both OPEN and CLOSED set (Algorithm 2 line 4-8). Collision query is performed between the neighbor and the current configuration (Algorithm 2 line 9). The actual cost of the neighbor would be calculated if there is a collision-free path from current configuration to successor configuration (Algorithm 2 line 10). If the newly calculated actual cost is smaller than the original actual cost, then the new configuration costs are updated and the successor configuration is inserted into the OPEN set (Algorithm 2 line 11-15). Finally, after all configurations in the neighbor set are expanded, the current

Algorithm 2: Expanding Process
 $\{S_{open}, S_{closed}\} = \text{Expand}(S_{open}, S_{closed}, p)$

```

1  $T_{EE} = f_{fk}(p \triangleq \theta_{cur})$ ;
2 foreach  $T_{suc} \in S_{neighbor}(T_{EE})$  do
3    $p_{suc} \triangleq \theta_{suc} = f_{ik}(T_{suc}, \theta_{cur})$ ;
4   if  $p_{suc}$  is in the CLOSED set then
5     continue;
6   else if  $p_{suc}$  is not in the OPEN set then
7      $f(p_{suc}) = g(p_{suc}) = \infty$ ;
8   end
9   if  $\text{CollisionFree}(p, p_{suc})$  then
10    Calculate the new actual cost  $g(p_{suc})_{new}$ 
    according to the equations (2), (4) and (5).
11    if  $g(p_{suc})_{new} < g(p_{suc})$  then
12      Update the actual cost:
       $g(p_{suc}) = g(p_{suc})_{new}$ ;
13      Update the total cost:  $f(p_{suc})$  according
      to the equation (1), (3) to (8);
14      Insert the neighbor configuration into the
      OPEN set:  $S_{open} = S_{open} \cup \{p_{suc}\}$ ;
15    end
16  end
17 end
18  $S_{closed} = S_{closed} \cup \{p\}$ ,  $S_{open} = S_{open} / \{p\}$ ;

```

configuration is moved from the OPEN set to the CLOSED set. The expanding process ends.

III. EXPERIMENTAL RESULTS

In this section, experiments are conducted not only to verify the proposed algorithm's ability of avoiding the singularity and joint limits, but also to demonstrate how our algorithms find the optimal path in the task redundant space. Welding which has many task redundancy DOFs and is usually deployed in a dense object environment is taken as an example throughout this experimental section. Simulations show that our proposed algorithm could solve the optimal motion planning problem with traditional industrial robot and could also be implemented to other task redundant applications. All experiments are carried out on the self-developed simulation platform.

A. Singularity and Joint Limits Avoidance

The first experiment is conducted to verify the algorithms' singularity avoidance ability. A scenario with a singularity along its trajectory is specially designed in the experiments. Fig. 1(a) shows the experiment scenario. The welding torch equipped at the end of the robot is moving along a horizontal trajectory. The singularity exists at the middle of the planned trajectory. Singularity costs are compared with two kinds of motion planning algorithms. Fig. 1(b) compares the singularity costs with and without considering the singularity. The results demonstrate that our proposed algorithm could return a path with lower singularity cost. Fig. 1(c)-1(d) show the

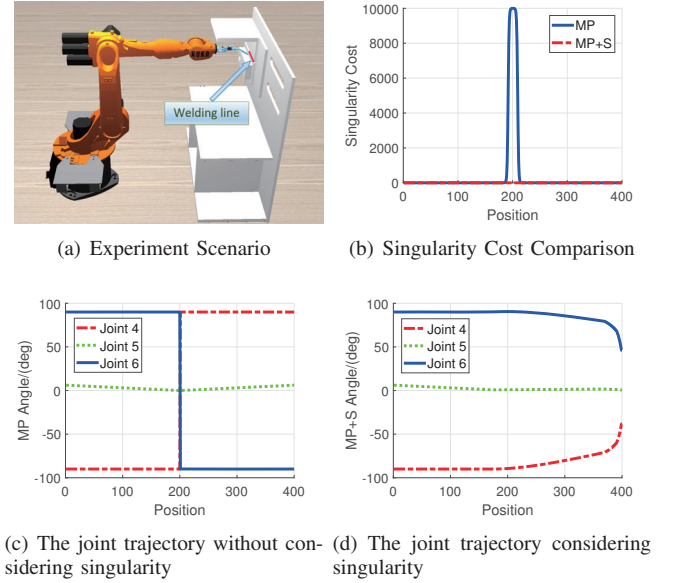


Fig. 1. Singularity Avoidance Experiment (MP stands for Motion planning without considering singularity, MP+S stands for Motion planning considering singularity.)

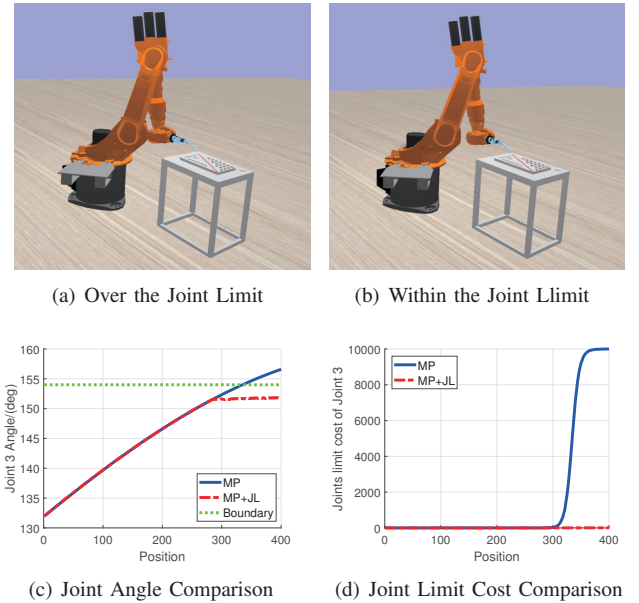


Fig. 2. Joint Limit Experiment (MP: Motion planning without considering the joint limits. MP+JL: Motion planning considering the joint limits.)

joint trajectories of the two algorithms. Algorithms without singularity avoidance would cause the 4th joint and the 6th joint jump at the singularity point. And the 5th joint would decrease to zero and turn backward at zero position without crossing. On the contrary, the changes in 4th and 6th joint angles will be relatively gentle, and the 5th joint will be close to zero position but not equal to zero when singularity is considered in the motion planning algorithm.

The second experiment is conducted to verify the joint limits avoidance ability of the proposed algorithm. Fig. 2 shows the comparison results of two different motion plan-

ning algorithms with and without considering the joint limits. The experiment is carried out with the same robot tracking the same welding seam on a plate. The only difference is that Fig. 2(a) shows the robot configuration that the angle of the joint exceeds the joint limits. If we could convert the joint angle limits to the cost, joint over limits could be avoided during motion planning. Fig. 2(b) shows the improved robot configuration with all joints within their boundaries. It is easy to find out that the robot posture looks more natural if joint limits are considered. Fig. 2(c) compares the 3rd joint trajectories under both situations. We could also see from the joint angle data that our proposed algorithm could constrain the joint angle within its boundaries under all circumstances. This would prevent the robot from occasionally shutting down and possibly causing some damage to itself. Our proposed algorithm avoids the potential problems by considering the joint limits cost in the motion planning. The cost variations when joint angle approaches the boundary is illustrated in Fig. 2(d). The joint limits cost will rise sharply as the joint approaches the boundary value. As we could see that our proposed motion planner would automatically choose the lower cost path which avoid the joint over limits.

B. Motion Planning with 1-D Task Redundancy

In this experiment, a specific welding task which aims to control the welding torch to track a given welding seam on a yellow plane with a magenta cylinder obstacle on one side is shown in Fig. 3(a). We assume that the welding torch should be perpendicular to the plane and could only rotate along its symmetrical axis, therefore the task redundant space is one-dimension. Fig. 3(b) shows the initial configuration of the robot. It could easily be noticed that simply following the torch trajectory would trigger collision between the cylinder obstacle and the robot. By implementing our proposed algorithm, robot would rotate along the symmetrical axis of the welding torch to the opposite side of the cylinder. From Fig. 3(c) and 3(d), we could clearly see the process of obstacle avoidance.

We could also analyse the motion planning algorithm by zoning the task redundant space. Fig. 4 marks different types of areas in the task redundant space with different colors. The yellow areas are the feasible configurations. The blue areas represent the configurations which would collide with the obstacles. The green areas indicate the unreachable configurations, and configurations which are out of joint limits are marked with magenta. Some areas have multiple features, such as the orange areas indicate that the configurations are both collided and beyond the joint limits. The singularity area doesn't exist in this experiment, but the representation is the same with the other area if exists. The problem in this experiment is to find an optimal trajectory in the feasible zone within this colored task redundant space. The red line indicates the optimal trajectory result. We could notice that our algorithm is able to find the optimal trajectory in the task redundant space.

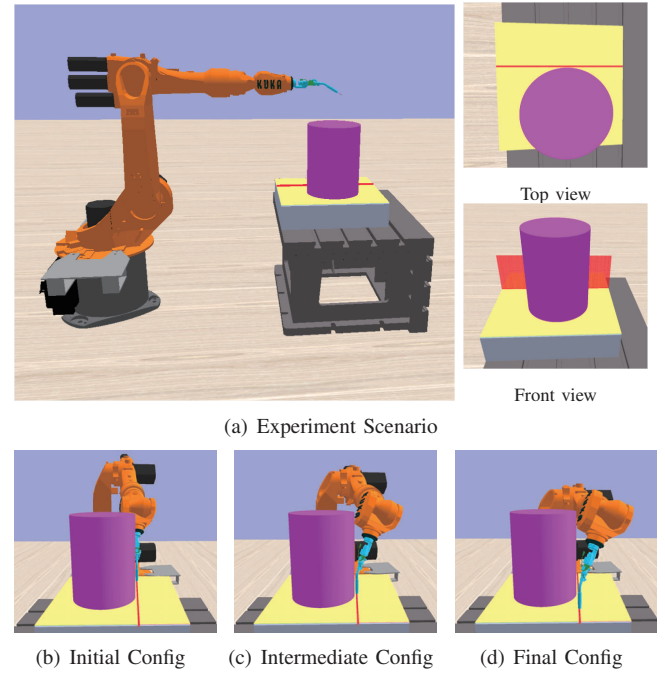


Fig. 3. Motion Planning with 1-D Task Redundancy

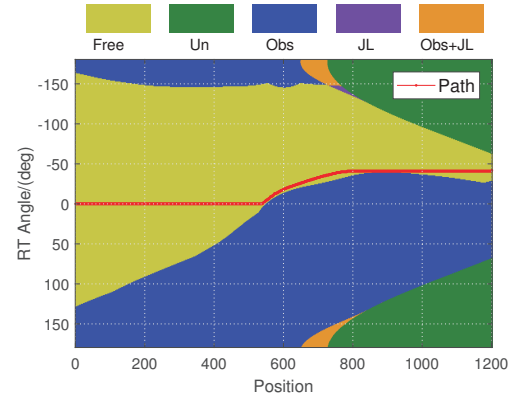


Fig. 4. Different zones of the task redundant space. (Free: Feasible areas; Un: Unreachable areas; Obs: Obstacle areas; JL: Joint limits areas; RT: Rotation around tool.)

C. Motion Planning with 2-D Task Redundancy

Robotic intersecting pipes welding in pipeline engineering and pressure vessel manufacturing is a typical example of complex robotic trajectory planning. In this example, the welding torch is constrained to the intersection line of two pipes with two degrees of freedom. One is the symmetrical axis of the torch and the other is the swing angle. Fig. 5(a) shows the experiment scenario. A cluster of red line segments represent the planned welding torch directions. From two welding snapshots shown on Fig. 5(b), we could easily find out that our proposed algorithm could generate a collision-free trajectory among complex environment with cluttered obstacles.

Since it is difficult and complex to depict the shapes of different regions and the trajectory in the two-dimensional

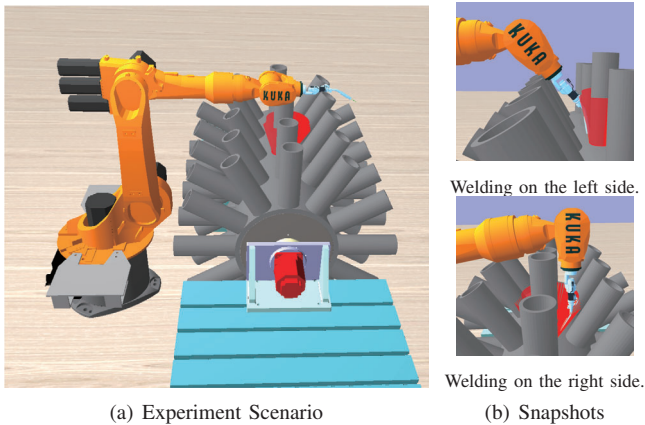


Fig. 5. Motion Planning with 2-D Task Redundancy

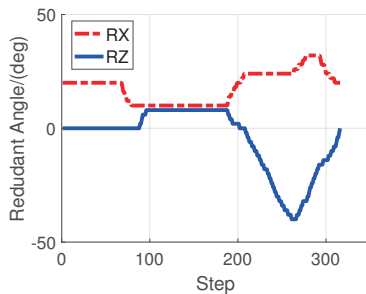


Fig. 6. The curves of redundant angles versus each steps. RZ curve stands for the rotational angle along the symmetrical axis of the welding torch and RX curve represents the rotational angle of the welding seam. Both singularity and joint limits are considered in this experiment.

task redundant space as the previous experiment, we plot the redundant angle's trajectory to specify how the motion planning algorithm works. Fig. 6 shows the curves of the redundant angles versus each steps. From the results we could see that our proposed motion planning algorithm plays a very important role in the complex environment with dense obstacles. It also should be noted that the redundant DOFs are no longer need to be concerned by using our proposed algorithm, which would make the motion planning much easier for most task redundant applications.

IV. CONCLUSION

Task redundancy exists in most robotic applications. An optimal motion planning algorithm is proposed to automatically generate a collision-free trajectory of typical industrial robot in applications where limited constraints are given. The A* algorithm which is originally designed for grid is modified to be implemented on traditional industrial robot. The cost evaluation is elaborately designed to consider not only distance in task redundant space but also the cost of singularity and joint limits. The experiment results verify that our proposed algorithm not only could handle the singularity and joint limits, realize a collision-free trajectory, but also could be used in complex environment.

REFERENCES

- [1] S. Xia, C. K. Pang, A. Al Mamun, C. M. Chew, and K. P. Tan, "Feedforward compensation for suppression of seam boundary error propagation in robotic welding systems," *IEEE/ASME Transactions on Mechatronics*, vol. 23, no. 4, pp. 1919–1929, 2018.
- [2] J. Liu, X. Huang, S. Fang, H. Chen, and N. Xi, "Industrial robot path planning for polishing applications," in *2016 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2016, pp. 1764–1769.
- [3] B. Zhang, J. Wu, L. Wang, Z. Yu, and P. Fu, "A method to realize accurate dynamic feedforward control of a spray-painting robot for airplane wings," *IEEE/ASME Transactions on Mechatronics*, vol. 23, no. 3, pp. 1182–1192, 2018.
- [4] Z. Zhang, Y. Lin, S. Li, Y. Li, Z. Yu, and Y. Luo, "Tricriteria optimization-coordination motion of dual-redundant-robot manipulators for complex path planning," *IEEE Transactions on Control Systems Technology*, vol. 26, no. 4, pp. 1345–1357, 2017.
- [5] Z. Li, C. Li, S. Li, and X. Cao, "A fault-tolerant method for motion planning of industrial redundant manipulator," *IEEE Transactions on Industrial Informatics*, 2019.
- [6] W. Lin, A. Anwar, Z. Li, M. Tong, J. Qiu, and H. Gao, "Recognition and pose estimation of auto parts for an autonomous spray painting robot," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 3, pp. 1709–1719, 2019.
- [7] N. Larkin, A. Short, Z. Pan, and S. van Duin, "Automated programming for robotic welding," in *Transactions on Intelligent Welding Manufacturing*. Springer, 2018, pp. 48–59.
- [8] H. Cui, Z. Jiang, X. Tang, and F. Lu, "Research on narrow-gap gmaw with swing arc system in horizontal position," *The International Journal of Advanced Manufacturing Technology*, vol. 74, no. 1-4, pp. 297–305, 2014.
- [9] D. Guo, Z. Li, A. H. Khan, Q. Feng, and J. Cai, "Repetitive motion planning of robotic manipulators with guaranteed precision," *IEEE Transactions on Industrial Informatics*, 2020.
- [10] J.-J. Kim and J.-J. Lee, "Trajectory optimization with particle swarm optimization for manipulator motion planning," *IEEE Transactions on Industrial Informatics*, vol. 11, no. 3, pp. 620–631, 2015.
- [11] I. Streinu, "Combinatorial roadmaps in configuration spaces of simple planar polygons," in *Proc. DIMACS Workshop, Algorithmic and Quantitative Aspects of Real Algebraic Geometry in Mathematics and Computer Science*, 2003, pp. 181–206.
- [12] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Autonomous robot vehicles*. Springer, 1986, pp. 396–404.
- [13] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [14] S. Koenig, M. Likhachev, and D. Furcy, "Lifelong planning a*," *Artificial Intelligence*, vol. 155, no. 1-2, pp. 93–146, 2004.
- [15] M. Likhachev, D. I. Ferguson, G. J. Gordon, A. Stentz, and S. Thrun, "Anytime dynamic a*: An anytime, replanning algorithm," in *ICAPS*, vol. 5, 2005, pp. 262–271.
- [16] M. Likhachev, G. J. Gordon, and S. Thrun, "Ara*: Anytime a* with provable bounds on sub-optimality," in *Advances in neural information processing systems*, 2004, pp. 767–774.
- [17] L. E. Kavrakı, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [18] S. M. LaValle and J. J. Kuffner Jr, "Randomized kinodynamic planning," *The international journal of robotics research*, vol. 20, no. 5, pp. 378–400, 2001.
- [19] S. M. LaValle, M. S. Branicky, and S. R. Lindemann, "On the relationship between classical grid search and probabilistic roadmaps," *The International Journal of Robotics Research*, vol. 23, no. 7-8, pp. 673–692, 2004.
- [20] T. Yoshikawa, "Manipulability of robotic mechanisms," *The international journal of Robotics Research*, vol. 4, no. 2, pp. 3–9, 1985.
- [21] A. Goldenberg, B. Benhabib, and R. Fenton, "A complete generalized solution to the inverse kinematics of robots," *IEEE Journal on Robotics and Automation*, vol. 1, no. 1, pp. 14–20, 1985.